

LEGAL EASE

AI-Powered Legal Document Generator

COMPLETE PROJECT DOCUMENTATION

1. PROJECT TITLE

LegalEase: AI-Powered Legal Document Generator

LegalEase leverages Generative AI to simplify the creation of legal documents by providing customizable, accurate, and editable templates for a wide range of use cases.

2. PROJECT DESCRIPTION

Users can generate employment contracts, lease agreements, NDAs, and other legal documents based on inputs such as involved parties, effective dates, and key terms. The system uses FastAPI and an AI core, provides professional formatting, custom branding, editable previews, automatic term tables, secure handling of sensitive user data, and export options in PDF, DOCX, and TXT formats.

3. SCENARIOS / USE CASES

Scenario 1 – Employment Contract

A startup founder enters details about a new hire and selects Employment Contract. LegalEase generates an agreement covering roles, responsibilities, compensation, and confidentiality. The user can customize wording, add a company logo, and download a branded PDF.

Scenario 2 – NDA

A freelancer specifies the parties, confidentiality scope, and effective date to generate a structured NDA.

Scenario 3 – Lease Agreement

A landlord provides property address, tenant details, and lease terms. The system creates a formatted lease agreement with editable sections and DOCX export.

4. PRE-REQUISITES

- FastAPI framework knowledge
- Google Gemini API familiarity
- Streamlit knowledge for the frontend
- Python programming proficiency
- Environment setup with required packages such as fastapi, uvicorn, streamlit, python-docx, fpdf, Pillow, requests, google-generativeai, and python-dotenv.

5. PROJECT WORKFLOW

Activity 1 – Model Selection and System Architecture

Gemini 1.5 Pro is selected for legal document generation. The architecture uses Streamlit as frontend, FastAPI as backend, Gemini integration as the AI core, and document-formatting modules for DOCX, PDF, and TXT.

Activity 2 – Core Functionalities Development

The project implements legal document creation, editable text preview, and export as TXT, DOCX, and PDF. The /generate endpoint processes document type, parties, terms, and dates.

Activity 3 – main.py & API Logic

main.py initializes FastAPI, defines the root route, and includes routes.py. routes.py defines DocumentRequest and calls GeminiDocumentGenerator.generate_document().

Activity 4 – Frontend with Streamlit

The frontend provides required input fields, responsive layout, dynamic document preview, editing, and export options.

Activity 5 – Deployment

Deployment preparation includes production-ready dependencies, requirements.txt, Procfile or Dockerfile if needed, and possible deployment platforms described in the source documentation.

Activity 6 – Conclusion / Final Testing

The project is finalized by checking end-to-end functionality and testing Gemini integration with different input combinations.

6. SYSTEM ARCHITECTURE

The system consists of four main parts: Streamlit frontend, FastAPI backend, Gemini AI core, and document-formatting modules. Users provide document details through the frontend. FastAPI receives the request and routes it to GeminiDocumentGenerator. Gemini returns generated content, which is displayed to the user and can then be edited or exported.

7. FRONTEND FUNCTIONALITY

The frontend accepts document type and dates through text inputs and parties and terms through text areas. It provides Generate Document, Edit Document, and Download as TXT/DOCX/PDF options. A styled HTML preview displays the generated legal document.

8. BACKEND RESPONSIBILITIES

FastAPI exposes the POST /generate endpoint, accepts JSON input containing document_type, parties, terms, and dates, routes the request to GeminiDocumentGenerator, and returns structured document text.

9. AI INTEGRATION

GeminiDocumentGenerator forms a structured prompt and sends it to Gemini through generate_content. The generated text is returned to FastAPI and displayed by the frontend for preview, editing, and exporting.

10. CORE FUNCTIONALITIES

Legal Document Generation

Generates legal documents based on document type, parties, terms, and dates.

Document Output Formatting

Converts AI-generated text into structured DOCX, PDF, and TXT files. DOCX formatting includes logo, Times New Roman font, terms table, and footer. PDF formatting includes headers, footers, logo, headings, and bullet-style terms.

User Interaction

Accepts user preferences through the Streamlit interface and reflects them in AI-generated legal content.

Editable Document & Download

Provides an editable text area and download options in TXT, DOCX, and PDF formats.

11. CORE UTILITY FUNCTIONS

sanitize_text(text)

Removes special characters and typographic quotes to ensure clean formatting.

format_docx(text, doc_type)

Uses python-docx to embed the logo, add titles and paragraph formatting, and generate a terms table from semicolon-separated input.

format_pdf(text, doc_type)

Uses FPDF to create a formatted document with a centered logo, bold section headings, and bullet-style terms.

format_html_preview(text)

Converts generated output into styled HTML blocks for inline display.

12. USER INPUTS

Document Type

Examples include Agreement, Contract, NDA, Lease Agreement, and Employment Offer Letter.

Parties Involved

Names and roles of individuals or entities involved in the document.

Terms & Conditions

Specific clauses or rules. The source documentation specifies separating bullet points using semicolons.

Effective Date

The date on which the agreement becomes legally valid.

13. DOCUMENT EDITING

After generation, users can select the Edit Document option and modify the generated content in an editable text area.

14. DOCUMENT EXPORT

LegalEase supports three output formats:

- TXT – plain text output.
- DOCX – formatted Word document with logo, headings, table, and footer.
- PDF – formatted document with logo and footer on pages.

15. DEPLOYMENT

The source documentation describes local deployment using FastAPI as backend and Streamlit as frontend. A virtual environment is used, dependencies are installed from requirements.txt, and the Gemini API key is stored in a .env file. The backend is launched with Uvicorn and the Streamlit application provides the UI.

16. TESTING & VERIFICATION

Testing includes entering document type, parties, terms, and date; generating content through Gemini; checking HTML preview; editing the generated text; downloading TXT, DOCX, and PDF; and checking layout, logo, footer, bullet formatting, and terms embedding. The source also specifies testing Gemini integration under different input combinations.

17. EXPECTED FINAL OUTPUT

The completed system provides an AI-assisted legal document generation workflow from user input to generated content, editable preview, and downloadable document formats.

18. CONCLUSION

LegalEase demonstrates the integration of Generative AI, backend APIs, document processing, and a user-friendly interface for legal document generation. The source documentation states that the platform is intended to improve accessibility to legal information and support clear, structured, and formatted legal content.

19. FUTURE ENHANCEMENTS

The source documentation identifies possible future enhancements including deeper contract analysis, integration with legal databases, and personalized recommendations.

Prepared from the supplied LegalEase project documentation.